

Apuntes de Analisis Numerico con implementacion en Python

Estos apuntes condensan el material de `material_apuntes`: teoria esencial, formulas que hay que saber manejar e implementaciones Python listas para reutilizar en practicas.

Indice

1. Errores y aritmetica de precision finita
 2. Ecuaciones no lineales
 3. Derivacion numerica
 4. Ecuaciones diferenciales ordinarias
 5. Diferenciacion automatica
 6. Integracion numerica
 7. Mapa de scripts del material
-

1. Errores y aritmetica de precision finita

En calculo numerico casi nunca trabajamos con la solucion exacta. Hay dos fuentes de error principales:

- **Error de redondeo:** aparece porque el ordenador representa los reales con precision finita.
- **Error de discretizacion o truncamiento:** aparece al sustituir un problema continuo por una aproximacion finita.

Si x es el valor exacto y x_a una aproximacion:

$$e_{\text{abs}} = |x - x_a|, \quad e_{\text{rel}} = \frac{|x - x_a|}{|x|}$$

El error absoluto mide la desviacion en unidades originales. El error relativo mide la desviacion en proporcion al tamaño de la magnitud.

```
def error_absoluto(x_exacto, x_aprox):
    return abs(x_exacto - x_aprox)

def error_relativo(x_exacto, x_aprox):
    if x_exacto == 0:
        raise ZeroDivisionError("El error relativo no esta definido si x_exacto = 0")
    return abs(x_exacto - x_aprox) / abs(x_exacto)
```

Precision finita

No todos los numeros reales se pueden representar exactamente. Por ejemplo:

```
print(1.1 + 0.1)
print(1.1 + 0.1 == 1.2)
```

La salida típica es `1.2000000000000002` y `False`.

El **epsilon de la maquina** es el menor número positivo `eps` tal que `1 + eps != 1` en la aritmética del ordenador.

```
def epsilon_maquina():
    x = 1.0
    while 1.0 + x != 1.0:
        x /= 2.0
    return 2.0 * x

print(epsilon_maquina())
```

Propagación de errores

Para sumas y restas se controlan mejor los errores absolutos:

$$\Delta(x \pm y) \lesssim \Delta x + \Delta y$$

Para productos y cocientes se controlan mejor los errores relativos:

$$\frac{\Delta(xy)}{|xy|} \lesssim \frac{\Delta x}{|x|} + \frac{\Delta y}{|y|}$$

La resta de números casi iguales es peligrosa: produce **cancelación catastrófica**.

Cancelación catastrófica en raíces de segundo grado

Para resolver

$$ax^2 + bx + c = 0$$

la fórmula clásica

$$x_{\pm} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

puede fallar cuando se resta `-b + sqrt(...)` y ambos términos son muy parecidos. Una forma más estable consiste en calcular primero la raíz que no cancela y luego usar `x_1 x_2 = c/a`.

```
from math import sqrt

def raices_cuadraticas_estables(a, b, c):
    d = sqrt(b * b - 4.0 * a * c)
```

```

if b >= 0:
    x2 = (-b - d) / (2.0 * a)
    x1 = c / (a * x2)
else:
    x1 = (-b + d) / (2.0 * a)
    x2 = c / (a * x1)

return x1, x2

print(raices_cuadraticas_estables(1.0, 1.0e8, 1.0))

```

Sumas largas y orden de acumulacion

El orden de suma importa. En aritmetica exacta la suma es asociativa; en coma flotante, no siempre.

```

x = 1.0e20
y = -1.0e20
z = 1.0

print((x + y) + z)  # 1.0
print(x + (y + z))  # 0.0

```

En el problema de Basilea,

$$\sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6},$$

sumar de menor a mayor suele reducir el error cuando hay terminos de tamanos muy distintos.

```

from math import pi

def basilea_hacia_delante():
    k = 1.0
    s = 0.0

    while True:
        termino = 1.0 / (k * k)
        nuevo = s + termino
        if nuevo == s:
            break
        s = nuevo
        k += 1.0

    return int(k) - 1, s

```

```
def basilea_hacia_atras(n):
    s = 0.0
    for k in range(n, 0, -1):
        s += 1.0 / (k * k)
    return s

m, s = basilea_hacia_delante()
valor_real = pi**2 / 6.0
print(m, s, abs(s - valor_real))
print(basilea_hacia_atras(10 * m))
```

2. Ecuaciones no lineales

Queremos resolver

$$f(x) = 0.$$

Un metodo iterativo genera aproximaciones x_k y se detiene cuando se cumple algun criterio como:

$$|x_{k+1} - x_k| < \varepsilon, \quad |f(x_k)| < \varepsilon.$$

Para evitar bucles infinitos se fija siempre un numero maximo de iteraciones.

Orden de convergencia

Si x^* es la raiz y $e_k = |x_k - x^*|$, el metodo tiene orden r si existe una constante $C > 0$ tal que

$$e_{k+1} \approx C e_k^r.$$

Experimentalmente, con tres errores consecutivos:

$$r \approx \frac{\log(e_{k-1}/e_k)}{\log(e_{k-2}/e_{k-1})}.$$

```
from math import log

def orden_convergencia(e1, e2, e3):
    return log(e2 / e3) / log(e1 / e2)
```

Metodo de biseccion

Requisitos:

- f continua en $[a, b]$.
- Cambio de signo: $f(a) f(b) < 0$.

Por el teorema de Bolzano existe al menos una raiz en (a, b) . En cada paso:

$$c_k = \frac{a_k + b_k}{2}.$$

Si $f(a_k) \cdot f(c_k) < 0$, la raíz está en $[a_k, c_k]$; si no, está en $[c_k, b_k]$. La longitud del intervalo se divide entre dos en cada iteración. Por eso el método converge siempre, pero solo linealmente.

Una cota habitual del error es:

$$|c_k - x^*| \leq \frac{b - a}{2^{k+1}}.$$

```
def biseccion(f, a, b, tol=1.0e-6, maxit=200, verbose=False):
    fa = f(a)
    fb = f(b)

    if fa * fb >= 0:
        raise ValueError("Biseccion requiere f(a) * f(b) < 0")

    if verbose:
        print("k\tc\t\tlongitud")

    for k in range(maxit):
        c = 0.5 * (a + b)
        fc = f(c)

        if verbose:
            print(f"{k}\t{c:.12g}\t{b - a:.3e}")

        if fc == 0.0 or (b - a) < tol:
            return c

        if fa * fc < 0:
            b = c
            fb = fc
        else:
            a = c
            fa = fc

    raise RuntimeError(f"Numero maximo de iteraciones alcanzado: {maxit}")
```

Ejemplo:

```
from math import log

f = lambda x: log(x) + x
raiz = biseccion(f, 0.1, 1.0, tol=1.0e-8, verbose=True)
print(raiz, f(raiz))
```

Numero de iteraciones necesarias para garantizar error menor que ϵ :

```
from math import ceil, log

def iteraciones_biseccion(a, b, eps):
    return ceil(log((b - a) / (2.0 * eps)) / log(2.0))
```

Iteracion de punto fijo

Se transforma la ecuacion $f(x) = 0$ en

$$x = g(x).$$

La iteracion es:

$$x_{k+1} = g(x_k).$$

La eleccion de g es decisiva. Si $|g'(x^*)| < 1$ cerca del punto fijo, la iteracion tiende a ser atractora. Si $|g'(x^*)| > 1$, normalmente se aleja.

```
def punto_fijo(g, x0, tol=1.0e-6, maxit=100, verbose=False):
    x = x0

    if verbose:
        print("k\tx_k\tterror")

    for k in range(maxit):
        x_next = g(x)
        error = abs(x_next - x)

        if verbose:
            print(f"{k}\t{x_next:.12g}\t{error:.3e}")

        if error < tol:
            return x_next

        x = x_next

    raise RuntimeError(f"Numero maximo de iteraciones alcanzado: {maxit}")
```

Ejemplo con la raiz positiva de $x^2 - x - 1 = 0$:

```
from math import sqrt

f = lambda x: x**2 - x - 1.0
g = lambda x: sqrt(x + 1.0)
```

```
raiz = punto_fijo(g, x0=1.0, tol=1.0e-8)
print(raiz, f(raiz))
```

Metodo de Newton

Newton aproxima f por la recta tangente en x_k y toma como siguiente aproximacion la raiz de esa recta:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Si la raiz es simple, $f'(x^*) \neq 0$, y x_0 esta suficientemente cerca, la convergencia es cuadratica.

```
def newton(f, df, x0, tol=1.0e-6, maxit=100, verbose=False):
    x = x0

    if verbose:
        print("k\tx_k\tterror")

    for k in range(maxit):
        dfx = df(x)
        if dfx == 0.0:
            raise ZeroDivisionError("Newton no puede dividir por f'(x_k) = 0")

        x_next = x - f(x) / dfx
        error = abs(x_next - x)

        if verbose:
            print(f"{k}\t{x_next:.12g}\t{error:.3e}")

        if error < tol:
            return x_next

        x = x_next

    raise RuntimeError(f"Numero maximo de iteraciones alcanzado: {maxit}")
```

Ejemplo:

```
from math import log

f = lambda x: log(x) + x
df = lambda x: 1.0 / x + 1.0

raiz = newton(f, df, x0=1.0, tol=1.0e-10)
print(raiz, f(raiz))
```

Metodo de la secante

La secante evita calcular f' . Sustituye la derivada de Newton por una diferencia finita:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

La iteracion queda:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Su convergencia es superlineal, con orden teorico cercano al numero aureo:

$$r = \frac{1 + \sqrt{5}}{2} \approx 1.618.$$

```
def secante(f, x_prev, x, tol=1.0e-6, maxit=100, verbose=False):
    f_prev = f(x_prev)
    fx = f(x)

    if verbose:
        print("k\tx_k\terror")
        print(f"-1\t{x_prev:.12g}")
        print(f"0\t{x:.12g}")

    for k in range(1, maxit + 1):
        denom = fx - f_prev
        if denom == 0.0:
            raise ZeroDivisionError("Secante: f(x_k) - f(x_{k-1}) = 0")

        x_next = x - fx * (x - x_prev) / denom
        error = abs(x_next - x)

        if verbose:
            print(f"{k}\t{x_next:.12g}\t{error:.3e}")

        if error < tol:
            return x_next

        x_prev, x = x, x_next
        f_prev, fx = fx, f(x)

    raise RuntimeError(f"Numero maximo de iteraciones alcanzado: {maxit}")
```

Ejemplo:

```
from math import exp

f = lambda x: x * (exp(x / 2.0) + 1.0)
```



```
raiz = secante(f, 2.5, 2.0, tol=1.0e-12, maxit=50, verbose=True)
print(raiz, f(raiz))
```

Version con estimacion experimental del orden:

```
from math import log

def secante_con_orden(f, x_prev, x, x_star, tol=1.0e-12, maxit=100):
    errores = []
    f_prev = f(x_prev)
    fx = f(x)

    for _ in range(maxit):
        x_next = x - fx * (x - x_prev) / (fx - f_prev)
        errores.append(abs(x_next - x_star))

        if len(errores) >= 3:
            e1, e2, e3 = errores[-3], errores[-2], errores[-1]
            r = log(e2 / e3) / log(e1 / e2)
            print(f"x = {x_next:.12g}, orden aprox = {r:.6f}")

        if abs(x_next - x) < tol:
            return x_next

        x_prev, x = x, x_next
        f_prev, fx = fx, f(x)

    raise RuntimeError(f"Numero maximo de iteraciones alcanzado: {maxit}")
```

3. Derivacion numerica

Queremos aproximar $f'(x)$ usando valores de f . Las diferencias finitas se obtienen con desarrollos de Taylor.

Diferencias finitas de primer orden

Diferencia progresiva:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \quad E = O(h).$$

Diferencia regresiva:

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}, \quad E = O(h).$$

Diferencia centrada:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \quad E = O(h^2).$$

```
def calc_fd(f, x, h):
    """Diferencia progresiva."""
    return (f(x + h) - f(x)) / h

def calc_bd(f, x, h):
    """Diferencia regresiva."""
    return (f(x) - f(x - h)) / h

def calc_cd(f, x, h):
    """Diferencia centrada."""
    return (f(x + h) - f(x - h)) / (2.0 * h)
```

Ejemplo de comparacion:

```
from math import sin, cos, pi

f = lambda x: x * sin(x)
df = lambda x: sin(x) + x * cos(x)
x = pi / 2.0
exacto = df(x)

for h in [10.0**(-i) for i in range(1, 6)]:
    error_fd = abs(calc_fd(f, x, h) - exacto)
    error_bd = abs(calc_bd(f, x, h) - exacto)
    error_cd = abs(calc_cd(f, x, h) - exacto)
    print(h, error_fd, error_bd, error_cd)
```

Error de redondeo frente a error de truncamiento

Si h es grande, domina el error de truncamiento. Si h es demasiado pequeño, restamos números casi iguales y domina el error de redondeo. Por eso existe un h óptimo.

Para la diferencia progresiva, el error total se comporta como:

$$E(h) \approx C_1 h + C_2 \frac{\varepsilon_{\text{maq}}}{h}.$$

Para la centrada:

$$E(h) \approx C_1 h^2 + C_2 \frac{\varepsilon_{\text{maq}}}{h}.$$

Segunda derivada

Sumando desarrollos de Taylor se obtiene:

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}, \quad E = O(h^2).$$

```
def calc_segunda_centrada(f, x, h):
    return (f(x + h) - 2.0 * f(x) + f(x - h)) / (h * h)
```

Extrapolacion de Richardson

Si una aproximacion tiene desarrollo

$$A(h) = A + Ch^p + O(h^{p+1}),$$

se combinan $A(h)$ y $A(h/2)$ para cancelar el termino dominante:

$$R(h) = \frac{2^p A(h/2) - A(h)}{2^p - 1}.$$

Aplicado a la diferencia progresiva, donde $p = 1$:

```
def richardson(aprox, h, p):
    return (2.0**p * aprox(h / 2.0) - aprox(h)) / (2.0**p - 1.0)

def calc_rfd(f, x, h):
    """Richardson sobre diferencia progresiva."""
    return 2.0 * calc_fd(f, x, h / 2.0) - calc_fd(f, x, h)

def calc_rcd(f, x, h):
    """Richardson sobre diferencia centrada."""
    return (4.0 * calc_cd(f, x, h / 2.0) - calc_cd(f, x, h)) / 3.0
```

Ejemplo:

```
from math import sin, cos, pi

f = lambda x: x * sin(x)
df = lambda x: sin(x) + x * cos(x)
x = pi / 2.0
exacto = df(x)

for h in [10.0**(-i) for i in range(1, 6)]:
    print(
        h,
        abs(calc_fd(f, x, h) - exacto),
        abs(calc_cd(f, x, h) - exacto),
        abs(calc_rfd(f, x, h) - exacto),
```

```
abs(calc_rcd(f, x, h) - exacto),
)
```

Derivadas sobre una malla

Si tenemos valores tabulados en una malla uniforme $x_i = a + i h$, la derivada se aproxima con diferencias centradas en los puntos interiores y diferencias laterales en los extremos.

```
import numpy as np

def derivada_malla(y, h):
    y = np.asarray(y, dtype=float)
    dy = np.empty_like(y)

    dy[0] = (y[1] - y[0]) / h
    dy[-1] = (y[-1] - y[-2]) / h
    dy[1:-1] = (y[2:] - y[:-2]) / (2.0 * h)

    return dy
```

4. Ecuaciones diferenciales ordinarias

Un problema de valor inicial de primer orden se escribe:

$$y'(t) = f(t, y(t)), \quad y(a) = y_0.$$

Discretizamos $[a, b]$ en nodos:

$$t_i = a + ih, \quad h = \frac{b - a}{N}.$$

Buscamos aproximaciones $y_i \approx y(t_i)$.

Metodo de Euler explicito

Sustituimos la derivada por una diferencia progresiva:

$$\frac{y_{i+1} - y_i}{h} \approx f(t_i, y_i).$$

Entonces:

$$y_{i+1} = y_i + hf(t_i, y_i).$$

Euler explicito tiene error local $O(h^2)$ y error global $O(h)$.

```
import numpy as np
```

```
def euler(f, t, y0):
    """
    Euler explicito para y' = f(t, y), y(t[0]) = y0.
    """
    t = np.asarray(t, dtype=float)
    y = np.zeros_like(t, dtype=float)
    h = (t[-1] - t[0]) / (len(t) - 1)

    y[0] = y0
    for i in range(len(t) - 1):
        y[i + 1] = y[i] + h * f(t[i], y[i])

    return y
```

Ejemplo $y' = y, y(0)=1$, solucion exacta $y(t)=e^t$:

```
import numpy as np

f = lambda t, y: y
a, b = 0.0, 2.0
N = 10
t = np.linspace(a, b, N + 1)
y = euler(f, t, y0=1.0)

print(y[-1], np.exp(b), abs(y[-1] - np.exp(b)))
```

Metodo de Euler implicito

Sustituimos la derivada por una diferencia regresiva:

$$\frac{y_{i+1} - y_i}{h} \approx f(t_{i+1}, y_{i+1}).$$

Entonces:

$$y_{i+1} = y_i + hf(t_{i+1}, y_{i+1}).$$

Es implicito porque y_{i+1} aparece a ambos lados. En general hay que resolver una ecuacion no lineal en cada paso.

Para el problema lineal $y' = \text{lambda } y$, se despeja:

$$y_{i+1} = \frac{y_i}{1 - h\lambda}.$$

```
def euler_implicito_test(lambda_, t, y0):
    """
    Euler implicito para y' = lambda*y.
    """
    t = np.asarray(t, dtype=float)
```

```

y = np.zeros_like(t, dtype=float)
h = (t[-1] - t[0]) / (len(t) - 1)

y[0] = y0
for i in range(len(t) - 1):
    y[i + 1] = y[i] / (1.0 - h * lambda_)

return y

```

Euler implicito tiene el mismo orden global que Euler explicito, $O(h)$, pero mejores propiedades de estabilidad para problemas rigidos.

Metodo de Heun

Heun, o Euler mejorado, usa una prediccion de Euler y corrige promediando pendientes:

$$k_1 = f(t_i, y_i), \quad k_2 = f(t_i + h, y_i + hk_1),$$

$$y_{i+1} = y_i + \frac{h}{2}(k_1 + k_2).$$

Su error local es $O(h^3)$ y su error global $O(h^2)$.

```

def heun(f, t, y0):
    """
    Metodo de Heun para y' = f(t, y), y(t[0]) = y0.
    """
    t = np.asarray(t, dtype=float)
    y = np.zeros_like(t, dtype=float)
    h = (t[-1] - t[0]) / (len(t) - 1)

    y[0] = y0
    for i in range(len(t) - 1):
        k1 = f(t[i], y[i])
        k2 = f(t[i] + h, y[i] + h * k1)
        y[i + 1] = y[i] + 0.5 * h * (k1 + k2)

    return y

```

Comparacion de errores globales:

```

import numpy as np

f = lambda t, y: y
a, b = 0.0, 2.0
y_real_b = np.exp(b)
Ns = [10, 20, 50, 100, 200, 500, 1000]

for N in Ns:

```

```
t = np.linspace(a, b, N + 1)
h = (b - a) / N
err_euler = abs(euler(f, t, 1.0)[-1] - y_real_b)
err_heun = abs(heun(f, t, 1.0)[-1] - y_real_b)
print(h, err_euler, err_heun)
```

Estabilidad

Para el problema test

$$y' = \lambda y, \quad \lambda < 0,$$

Euler explicito da:

$$y_{i+1} = (1 + h\lambda)y_i.$$

Para que la solucion numerica decaiga como la exacta necesitamos:

$$|1 + h\lambda| < 1.$$

Si $\lambda < 0$, esto equivale a:

$$0 < h < \frac{2}{|\lambda|}.$$

Euler implicito, en cambio:

$$y_{i+1} = \frac{1}{1 - h\lambda} y_i,$$

es estable para cualquier $h > 0$ en este problema test. Por eso se usa en problemas rigidos, aunque cada paso sea mas caro.

5. Diferenciacion automatica

La diferenciacion automatica calcula derivadas evaluando codigo, no manipulando simbolos ni tomando diferencias finitas. La idea basica del modo forward usa numeros duales.

Un numero dual es:

$$a + b\varepsilon, \quad \varepsilon^2 = 0.$$

Se interpreta como el par (valor, derivada). Por ejemplo, para derivar una funcion en x_0 , se evalua la funcion con el dual $(x_0, 1)$.

Las reglas de operaciones reproducen las reglas de derivacion:

$$(a, b) + (c, d) = (a + c, b + d)$$

$$(a, b)(c, d) = (ac, ad + bc)$$

$$\frac{(a, b)}{(c, d)} = \left(\frac{a}{c}, \frac{bc - ad}{c^2} \right)$$

Implementacion minima con numeros duales

```
from math import cos, exp, log, sin

class Dual:
    def __init__(self, real, dual=0.0):
        self.real = float(real)
        self.dual = float(dual)

    def __repr__(self):
        return f"Dual(real={self.real}, dual={self.dual})"

    @staticmethod
    def lift(x):
        return x if isinstance(x, Dual) else Dual(x, 0.0)

    def __add__(self, other):
        other = Dual.lift(other)
        return Dual(self.real + other.real, self.dual + other.dual)

    __radd__ = __add__

    def __sub__(self, other):
        other = Dual.lift(other)
        return Dual(self.real - other.real, self.dual - other.dual)

    def __rsub__(self, other):
        other = Dual.lift(other)
        return other - self

    def __mul__(self, other):
        other = Dual.lift(other)
        return Dual(
            self.real * other.real,
            self.real * other.dual + self.dual * other.real,
        )

    __rmul__ = __mul__

    def __truediv__(self, other):
        other = Dual.lift(other)
        return Dual(
            self.real / other.real,
            (self.dual * other.real - self.real * other.dual) /
            other.real**2,
        )

    def __rtruediv__(self, other):
        other = Dual.lift(other)
        return other / self
```



```
def __pow__(self, n):
    return Dual(self.real**n, n * self.real ** (n - 1) * self.dual)

def __neg__(self):
    return Dual(-self.real, -self.dual)
```

Funciones elementales:

```
def dsin(x):
    x = Dual.lift(x)
    return Dual(sin(x.real), cos(x.real) * x.dual)

def dcos(x):
    x = Dual.lift(x)
    return Dual(cos(x.real), -sin(x.real) * x.dual)

def dexp(x):
    x = Dual.lift(x)
    value = exp(x.real)
    return Dual(value, value * x.dual)

def dlog(x):
    x = Dual.lift(x)
    return Dual(log(x.real), x.dual / x.real)
```

Funcion auxiliar para derivar:

```
def derivada_ad(f, x0):
    resultado = f(Dual(x0, 1.0))
    return resultado.real, resultado.dual
```

Ejemplo:

```
def f(x):
    return x * dsin(x) + dlog(x)

valor, derivada = derivada_ad(f, 2.0)
print(valor, derivada)
```

La segunda componente es $f'(2)$ con precision cercana a la de maquina, sin elegir un paso h .

En la practica, librerias como JAX, PyTorch o TensorFlow generalizan esta idea a funciones vectoriales, gradientes, jacobianos y hessianos.

6. Integracion numerica

Queremos aproximar:

$$I = \int_a^b f(x) dx$$

por una suma finita:

$$I \approx \sum_{i=0}^n w_i f(x_i),$$

donde x_i son los nodos y w_i los pesos.

Regla del punto medio

Regla simple:

$$\int_a^b f(x) dx \approx (b - a) f\left(\frac{a + b}{2}\right).$$

Regla compuesta con N subintervalos:

$$h = \frac{b - a}{N}, \quad x_{i+1/2} = a + \left(i + \frac{1}{2}\right) h,$$

$$I \approx h \sum_{i=0}^{N-1} f(x_{i+1/2}).$$

```
import numpy as np

def punto_medio_compuesto(f, a, b, n):
    h = (b - a) / n
    xs = a + (np.arange(n) + 0.5) * h
    return h * np.sum([f(x) for x in xs])
```

Regla del trapecio

Regla simple:

$$\int_a^b f(x) dx \approx \frac{b - a}{2} (f(a) + f(b)).$$

Regla compuesta:

$$I \approx h \left[\frac{1}{2} f(x_0) + \sum_{i=1}^{N-1} f(x_i) + \frac{1}{2} f(x_N) \right].$$

Tiene error global $O(h^2)$.

```
def trapecio_compuesto(f, a, b, n):
    h = (b - a) / n
    xs = np.linspace(a, b, n + 1)
    ys = np.array([f(x) for x in xs])
    return h * (0.5 * ys[0] + np.sum(ys[1:-1]) + 0.5 * ys[-1])
```

Si ya tenemos datos tabulados:

```
def trapecio_datos(y, x=None, dx=1.0):
    y = np.asarray(y, dtype=float)
    if x is None:
        h = dx
        return h * (0.5 * y[0] + np.sum(y[1:-1]) + 0.5 * y[-1])

    x = np.asarray(x, dtype=float)
    return np.sum(0.5 * (x[1:] - x[:-1]) * (y[1:] + y[:-1]))
```

Regla de Simpson

Simpson simple usa tres nodos a , $(a+b)/2$, b :

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right].$$

Simpson compuesto exige que N sea par:

$$I \approx \frac{h}{3} \left[f(x_0) + 4 \sum_{\substack{i=1 \\ i \text{ impar}}}^{N-1} f(x_i) + 2 \sum_{\substack{i=2 \\ i \text{ par}}}^{N-2} f(x_i) + f(x_N) \right].$$

Tiene error global $O(h^4)$ y grado de exactitud 3.

```
def simpson_compuesto(f, a, b, n):
    if n % 2 != 0:
        raise ValueError("Simpson compuesto requiere n par")

    h = (b - a) / n
    xs = np.linspace(a, b, n + 1)
    ys = np.array([f(x) for x in xs])

    return (h / 3.0) * (
        ys[0]
```

```

    + 4.0 * np.sum(ys[1:-1:2])
    + 2.0 * np.sum(ys[2:-1:2])
    + ys[-1]
)

```

Version para datos tabulados en malla uniforme:

```

def simpson_datos(y, dx=1.0):
    y = np.asarray(y, dtype=float)
    n = len(y) - 1

    if n % 2 != 0:
        raise ValueError("Simpson requiere un numero par de paneles")

    return (dx / 3.0) * (
        y[0]
        + 4.0 * np.sum(y[1:-1:2])
        + 2.0 * np.sum(y[2:-1:2])
        + y[-1]
    )

```

Trabajo de una fuerza variable

Si solo tenemos la fuerza $F(x)$ tabulada, el trabajo se aproxima integrando los datos:

$$W = \int_a^b F(x) dx.$$

```

import numpy as np

x = np.arange(0.0, 1.5, 0.1)
fuerza = np.array([
    0.0, 0.45, 1.45, 2.3, 3.1,
    3.1, 3.1, 2.5, 1.1, 1.1,
    1.1, 0.8, 0.6, 0.3, 0.0,
])

trabajo = trapecio_datos(fuerza, x)
print(trabajo)

```

La regla del trapecio integra implícitamente la interpolante lineal a trozos que une los datos medidos.

Comparacion rapida

Regla	Nodos	Error compuesto	Comentario
Punto medio	centros de panel	$O(h^2)$	simple y eficiente

Regla	Nodos	Error compuesto	Comentario
Trapecio	extremos de panel	$O(h^2)$	natural con datos tabulados
Simpson	extremos y punto medio	$O(h^4)$	exige numero par de paneles
Gauss-Legendre	nodos optimos	alto para pocas evaluaciones	nodos no equiespaciados

7. Mapa de scripts del material

Los scripts de `material_apuntes` quedan conectados así con la teoría:

Script	Tema	Que implementa
<code>problemaA.py</code>	Errores	Cancelacion catastrofica en la formula cuadratica
<code>ProblemaB2B3.py</code>	Errores	Suma de Basilea y efecto del orden de suma
<code>biseccion.py</code>	Raices	Metodo de biseccion y cota de iteraciones
<code>biseccion_orden_convergencia.py</code>	Raices	Estimacion experimental del orden
<code>Lab2A_secante.py</code>	Raices	Metodo de la secante
<code>Lab2B_secante_orden_convergencia.py</code>	Raices	Orden de convergencia de la secante
<code>4_1_ProblemaA (1).py</code>	Derivacion	Diferencias progresiva, regresiva y centrada
<code>richardson.py</code>	Derivacion	Extrapolacion de Richardson
<code>euler.py</code>	EDOs	Euler explicito
<code>4_2_ProblemaAyB.py</code>	EDOs	Euler explicito, Heun y comparacion de errores